

2026-05-06-p1-f16-opentelemetry-integration

P1-F16 — OpenTelemetry Integration Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development. Steps use checkbox (- []) syntax for tracking.

Goal: Ship real, end-to-end OpenTelemetry **tracing + metrics** for the HelixCode CLI agent. F16 wires an OTel TracerProvider and MeterProvider into the CLI bootstrap, decorates the three central hot paths (LLM provider calls, tool registry execution, agent loop iterations) with span + counter + histogram instrumentation, and routes telemetry to one of three exporters selected at startup via standard OTel environment variables: **OTLP/gRPC** (production), **OTLP/HTTP** (proxy-friendly), or **stdout** (development). When no exporter env var is set the stack collapses to a no-op TracerProvider + MeterProvider with zero observable cost. A `/telemetry` slash command (`status / show / flush`) provides inspection and force-flush. **No cobra subcommand** (Q5=B). **No yaml/CLI flags** (Q4=B). **No logs-bridge** (Q1=B; deferred to F16.5).

Architecture: New `internal/telemetry/` package with `types.go` (TelemetryConfig + ExporterKind enum + DefaultBlockedAttributeKeys + error sentinels), `config.go` (env-var parsing + exporter selection), `provider.go` (TelemetryProvider construction + Tracer/Meter accessors + ForceFlush + Shutdown + Stats + ring buffer), `llm_instrumentation.go` (TracedLLMProvider decorator using Go struct embedding so only Generate/GenerateStream are overridden), `tool_instrumentation.go` (in-place InstrumentToolCall helper used inside ToolRegistry.Execute), `agent_instrumentation.go` (in-place InstrumentAgentIteration helper used inside BaseAgent.executeTaskWithLLM), `attribute_filter.go` (CONST-042 secret-attribute blocklist) — each paired with `_test.go`. New slash at `internal/commands/telemetry_command.go`. Two existing files get small additions: `internal/tools/registry.go` (telemetryProvider field + SetTelemetryProvider setter + 5-line in-place wrap of Execute) and `internal/agent/base_agent.go` (telemetryProvider field + SetTelemetryProvider setter + 5-line wrap of executeTaskWithLLM's LLM-call site). `cmd/cli/main.go` adds the env-var load, provider construction, decorator wiring, slash registration, and deferred shutdown.

Tech Stack: Go 1.26, testify v1.11, spf13/cobra v1.8 — already present. **NEW external deps** (all pinned to OTel v1.30.0; pure Go; no CGO):

<code>go.opentelemetry.io/otel</code>	v1.3
<code>go.opentelemetry.io/otel/sdk</code>	v1.3
<code>go.opentelemetry.io/otel/sdk/metric</code>	v1.3
<code>go.opentelemetry.io/otel/metric</code>	v1.3
<code>go.opentelemetry.io/otel/trace</code>	v1.3
<code>go.opentelemetry.io/otel/exporters/otlp/otlptrace</code>	v1.3
<code>go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc</code>	v1.3
<code>go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracehttp</code>	v1.3
<code>go.opentelemetry.io/otel/exporters/otlp/otlpmetric</code>	v1.3
<code>go.opentelemetry.io/otel/exporters/otlp/otlpmetric/otlpmetricgrpc</code>	v1.3
<code>go.opentelemetry.io/otel/exporters/otlp/otlpmetric/otlpmetrichttp</code>	v1.3

go.opentelemetry.io/otel/exporters/stdout/stdouttrace
go.opentelemetry.io/otel/exporters/stdout/stdoutmetric

v1.3
v1.3

Estimated transitive footprint: ~25 new modules in `go.sum` (notably `google.golang.org/grpc`, `google.golang.org/protobuf`, `go.opentelemetry.io/proto/otlp`, `github.com/cenkalti/backoff/v4`).

Spec: `docs/superpowers/specs/2026-05-06-p1-f16-opentelemetry-integration-design.md` (commit `bc07a96`)

Working directory for go commands: `helix_code/`. Git from meta-repo root.

Anti-bluff smoke (FULL 4-term applied to F16 surface):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|placeholder" \
    internal/telemetry internal/commands/telemetry_command.go \
    && echo BLUFF || echo clean
```

Must always print `clean`.

Anti-bluff hot zone: §5.2 of the spec — telemetry can degenerate into a no-op in four ways: (a) Tracer initialised but no span ever created; (b) spans created but BSP never flushes (deferred shutdown in wrong scope, or 0-deadline ctx); (c) Meter registered but `Counter.Add` never called; (d) `MeterProvider` hooked to wrong exporter so increments never reach the receiver. The five real-execution criteria (spans created, spans exported, metrics recorded, metrics exported, no secret-shaped attributes) are each tested with both unit assertions AND a Challenge phase. The Challenge harness includes an in-tree fake OTLP/HTTP receiver (a tiny `net/http.Server` that decodes the protobuf body and asserts ≥ 1 trace POST + ≥ 1 metric POST). Span-count-zero is a hard Challenge failure — absence-of-error is NEVER acceptable.

Why this is consequential: observability is a load-bearing SRE surface for any production agent. Most “telemetry on” defects look fine in compilation and naive tests but produce zero spans at the collector — silently. F16’s discriminating tests are: (i) the Challenge’s in-tree fake OTLP/HTTP receiver POST counter, and (ii) the unit-level `tracetest.SpanRecorder` assertion that a span named `llm.generate` actually appears after a real `Generate` call through the wrapped provider. Both must produce positive evidence; neither can be satisfied by absence-of-error.

Task list



P1-F16-T01 — bootstrap evidence + advance PROGRESS to F16



P1-F16-T02 — `go.mod`: add OTel v1.30.0 dep set + `go.sum` (TDD: failing import test)



P1-F16-T03 — `internal/telemetry/types.go`: `TelemetryConfig` + `ExporterKind` + `DefaultBlockedAttributeKeys` + error sentinels (TDD)



P1-F16-T04 — `internal/telemetry/config.go` + `attribute_filter.go`: env-var parsing + exporter selection + secret-attribute filter (TDD)



P1-F16-T05 — `internal/telemetry/provider.go`: `TelemetryProvider` construction (`TracerProvider` + `MeterProvider`; selects exporter; pre-built instruments; `ForceFlush/Shutdown`) (TDD with `tracetest.SpanRecorder` + manual metric reader)



P1-F16-T06 — `internal/telemetry/llm_instrumentation.go`: `TracedLLMProvider` decorator (TDD with `FakeLLMProvider`; assert span name, token counter, no prompt-body attribute)



P1-F16-T07 — `internal/telemetry/tool_instrumentation.go` + `internal/tools/registry.go` wrap: `InstrumentToolCall` helper + in-place wrap of `Execute` + `SetTelemetryProvider` (TDD)



P1-F16-T08 — `internal/telemetry/agent_instrumentation.go` + `internal/agent/base_agent.go` wrap: `InstrumentAgentIteration` helper + in-place wrap of `executeTaskWithLLM` + `SetTelemetryProvider` (TDD)



P1-F16-T09 — `/telemetry` slash command (`status` / `show` / `flush`) (TDD; nil-provider reports unavailable)



P1-F16-T10 — `main.go` wiring (`env-var` load + provider construct + decorator + setters + slash + deferred shutdown) + integration tests (stdout always; OTLP gRPC + HTTP gated)



P1-F16-T11 — Challenge harness with in-tree fake OTLP/HTTP receiver (5-phase: `STDOUT` + `FAKE-OTLP-HTTP` + `FILTER` + `NOOP` + `REAL-COLLECTOR`)



P1-F16-T12 — Feature 16 close-out + push 4 remotes non-force

Task 1: Bootstrap

Append F16 evidence section header (spec `bc07a96`), update `PROGRESS` current focus to F16, insert F16 task list (12 items) after F15's. Confirm `06_phase_1_evidence.md` has an F16 anchor.

Commit: `docs(P1-F16-T01): bootstrap Phase 1 / Feature 16 evidence + advance PROGRESS.`

Task 2: go.mod (OTel v1.30.0 dep set, TDD)

Files: modify `helix_code/go.mod`, regenerate `helix_code/go.sum`. New ephemeral file `helix_code/internal/telemetry/_deps_smoke_test.go` (TDD).

Run from `helix_code/`:

```
go get go.opentelemetry.io/otel@v1.30.0
go get go.opentelemetry.io/otel/sdk@v1.30.0
go get go.opentelemetry.io/otel/sdk/metric@v1.30.0
go get go.opentelemetry.io/otel/metric@v1.30.0
```

```

go get go.opentelemetry.io/otel/trace@v1.30.0
go get go.opentelemetry.io/otel/exporters/otlp/otlptrace@v1.30.0
go get go.opentelemetry.io/otel/exporters/otlp/otlptrace/
    otlptracegrpc@v1.30.0
go get go.opentelemetry.io/otel/exporters/otlp/otlptrace/
    otlptracehttp@v1.30.0
go get go.opentelemetry.io/otel/exporters/otlp/otlpmetric@v1.30.0
go get go.opentelemetry.io/otel/exporters/otlp/otlpmetric/
    otlpmetricgrpc@v1.30.0
go get go.opentelemetry.io/otel/exporters/otlp/otlpmetric/
    otlpmetrichttp@v1.30.0
go get go.opentelemetry.io/otel/exporters/stdout/
    stdouttrace@v1.30.0
go get go.opentelemetry.io/otel/exporters/stdout/
    stdoutmetric@v1.30.0
go mod tidy

```

Failing test FIRST(_deps_smoke_test.go):

```
package telemetry
```

```

import (
    "testing"

    "go.opentelemetry.io/otel"
    "go.opentelemetry.io/otel/sdk/trace"
    "go.opentelemetry.io/otel/sdk/metric"
    "go.opentelemetry.io/otel/exporters/stdout/stdouttrace"
    "go.opentelemetry.io/otel/exporters/stdout/stdoutmetric"
    "go.opentelemetry.io/otel/exporters/otlp/otlptrace/
        otlptracegrpc"
    "go.opentelemetry.io/otel/exporters/otlp/otlptrace/
        otlptracehttp"
    "go.opentelemetry.io/otel/exporters/otlp/otlpmetric/
        otlpmetricgrpc"
    "go.opentelemetry.io/otel/exporters/otlp/otlpmetric/
        otlpmetrichttp"
)

```

```

func TestOTelDepsLink(t *testing.T) {
    _ = otel.GetTracerProvider()
    _ = trace.NewTracerProvider
    _ = metric.NewMeterProvider
    _ = stdouttrace.New
    _ = stdoutmetric.New
    _ = otlptracegrpc.New
    _ = otlptracehttp.New
    _ = otlpmetricgrpc.New
}

```

```

    _ = otelmetrichttp.New
}

```

Test fails on red (missing imports), passes after `go get`. The `_deps_smoke_test.go` file is removed in T03 once types.go lands.

Commit: feat(P1-F16-T02): add OpenTelemetry v1.30.0 dep set + dep-link smoke test.

Task 3: types.go (TDD)

Files: new helix_code/internal/telemetry/types.go, new helix_code/internal/telemetry/types_test.go. Delete _deps_smoke_test.go.

Define: - ExporterKind string enum (ExporterNone, ExporterStdout, ExporterOTLPGRPC, ExporterOTLPHTTP). - TelemetryConfig struct (Enabled, Kind, Endpoint, Protocol, ServiceName, ResourceAttributes, Insecure, Timeout, BlockedAttributeKeys). - TelemetryStats struct (Kind, Endpoint, ServiceName, SpansStarted, SpansEnded, MetricsRecord, StartTime, LastFlushAt). - ExportedRecord struct (Type {span|metric}, Name, Attributes, Timestamp) for the ring buffer. - DefaultBlockedAttributeKeys exported var (per spec §3.3). - Error sentinels: ErrTelemetryDisabled, ErrUnknownExporterKind, ErrExporterUnreachable.

Failing tests FIRST:

```

func TestExporterKind_StringValues(t *testing.T) {
    require.Equal(t, "none",      string(ExporterNone))
    require.Equal(t, "stdout",    string(ExporterStdout))
    require.Equal(t, "otlp-grpc", string(ExporterOTLPGRPC))
    require.Equal(t, "otlp-http", string(ExporterOTLPHTTP))
}

func TestDefaultBlockedAttributeKeys_CoversCredentialKeys(t
    *testing.T) {
    for _, k := range []string{"api_key", "token", "bearer",
        "password",
        "secret", "authorization", "anthropic_api_key",
        "openai_api_key",
        "aws_access_key_id", "aws_secret_access_key"} {
        require.Contains(t, DefaultBlockedAttributeKeys, k)
    }
}

func TestDefaultBlockedAttributeKeys_CoversPromptBodyKeys(t
    *testing.T) {
    for _, k := range []string{"prompt", "prompt_body",
        "request_body", "response_body"} {
        require.Contains(t, DefaultBlockedAttributeKeys, k)
    }
}

```

```

}

func TestTelemetryConfig_DefaultServiceName_helixcode(t
    *testing.T) {
    c := &TelemetryConfig{}
    c.applyDefaults()
    require.Equal(t, "helixcode", c.ServiceName)
}

```

Subject: feat(P1-F16-T03): TelemetryConfig + ExporterKind + DefaultBlockedAttributeKeys.

Task 4: config.go + attribute_filter.go (TDD)

Files: new helix_code/internal/telemetry/config.go, new helix_code/internal/telemetry/config_test.go, new helix_code/internal/telemetry/attribute_filter.go, new helix_code/internal/telemetry/attribute_filter_test.go.

config.go exports: - LoadConfigFromEnv() (*TelemetryConfig, error) — reads OTEL_EXPORTER_OTLP_ENDPOINT, OTEL_EXPORTER_OTLP_PROTOCOL, OTEL_SERVICE_NAME, OTEL_RESOURCE_ATTRIBUTES, OTEL_EXPORTER_OTLP_INSECURE, OTEL_EXPORTER_OTLP_TIMEOUT. When all are unset, returns &TelemetryConfig{Enabled: false, Kind: ExporterNone}. Returns ErrUnknownExporterKind on bogus protocol. - (*TelemetryConfig).SelectExporter() ExporterKind — applies the disambiguation rules from spec §3.4 (stdout > none-when-no-env > http/protobuf > grpc-default). - parseResourceAttributes(s string) map[string]string — k=v, k=v parser per Otel spec.

attribute_filter.go exports: - FilterAttributes(attrs []attribute.KeyValue, blocked []string) []attribute.KeyValue — drops KVs whose key matches any blocked entry (case-insensitive, exact match). - IsBlockedKey(key string, blocked []string) bool — used in tests.

Failing tests FIRST (config_test.go):

```

func TestLoadConfigFromEnv_NoEnv_DisablesTelemetry(t *testing.T)
{
    t.Setenv("OTEL_EXPORTER_OTLP_ENDPOINT", "")
    t.Setenv("OTEL_EXPORTER_OTLP_PROTOCOL", "")
    t.Setenv("OTEL_SERVICE_NAME", "")
    cfg, err := LoadConfigFromEnv()
    require.NoError(t, err)
    require.False(t, cfg.Enabled)
    require.Equal(t, ExporterNone, cfg.Kind)
}

func TestLoadConfigFromEnv_EndpointOnly_DefaultsOTLPGRPC(t
    *testing.T) {

```

```

t.Setenv("OTEL_EXPORTER_OTLP_ENDPOINT", "localhost:4317")
t.Setenv("OTEL_EXPORTER_OTLP_PROTOCOL", "")
cfg, err := LoadConfigFromEnv()
require.NoError(t, err)
require.True(t, cfg.Enabled)
require.Equal(t, ExporterOTLPGRPC, cfg.Kind)
}

func TestLoadConfigFromEnv_ProtocolStdout_SelectsStdout(t
    *testing.T) {
t.Setenv("OTEL_EXPORTER_OTLP_PROTOCOL", "stdout")
cfg, err := LoadConfigFromEnv()
require.NoError(t, err)
require.Equal(t, ExporterStdout, cfg.Kind)
}

func TestLoadConfigFromEnv_ProtocolHTTP_SelectsOTLPHTTP(t
    *testing.T) {
t.Setenv("OTEL_EXPORTER_OTLP_ENDPOINT", "http://
    localhost:4318")
t.Setenv("OTEL_EXPORTER_OTLP_PROTOCOL", "http/protobuf")
cfg, err := LoadConfigFromEnv()
require.NoError(t, err)
require.Equal(t, ExporterOTLPHTTP, cfg.Kind)
}

func TestLoadConfigFromEnv_BadProtocol_ReturnsErrUnknownExporterKind(t
    *testing.T) {
t.Setenv("OTEL_EXPORTER_OTLP_PROTOCOL", "carrier-pigeon")
_, err := LoadConfigFromEnv()
require.ErrorIs(t, err, ErrUnknownExporterKind)
}

func TestLoadConfigFromEnv_ResourceAttributesParsedAsCSV(t
    *testing.T) {
t.Setenv("OTEL_EXPORTER_OTLP_PROTOCOL", "stdout")
t.Setenv("OTEL_RESOURCE_ATTRIBUTES",
    "deployment.environment=prod,team=platform")
cfg, _ := LoadConfigFromEnv()
require.Equal(t, "prod",
    cfg.ResourceAttributes["deployment.environment"])
require.Equal(t, "platform", cfg.ResourceAttributes["team"])
}

```

Failing tests FIRST (attribute_filter_test.go):

```

func TestFilterAttributes_DropsBlockedKeys(t *testing.T) {
in := []attribute.KeyValue{
    attribute.String("api_key", "sk-abc"),

```

```

        attribute.String("token", "ghp_def"),
        attribute.String("password", "hunter2"),
        attribute.String("prompt", "leaked prompt body"),
        attribute.String("model", "gpt-4"),
    }
    out := FilterAttributes(in, DefaultBlockedAttributeKeys)
    require.Len(t, out, 1)
    require.Equal(t, "model", string(out[0].Key))
}

func TestFilterAttributes_CaseInsensitive(t *testing.T) {
    in := []attribute.KeyValue{
        attribute.String("API_KEY", "1"),
        attribute.String("Api-Key", "2"),
        attribute.String("apikey", "3"),
    }
    out := FilterAttributes(in, []string{"api_key", "api-key", "apikey"})
    require.Empty(t, out)
}

func TestFilterAttributes_NilBlocked_PassesThrough(t *testing.T) {
    in := []attribute.KeyValue{attribute.String("api_key", "x")}
    require.Equal(t, in, FilterAttributes(in, nil))
}

```

Subject: feat(P1-F16-T04): env-var config parsing + exporter selection + CONST-042 attribute filter.

Task 5: provider.go (TDD)

Files: new helix_code/internal/telemetry/provider.go, new helix_code/internal/telemetry/provider_test.go.

Implementation outline:

```

type TelemetryProvider struct {
    cfg                *TelemetryConfig
    tracerProv         *sdktrace.TracerProvider
    meterProv          *sdkmetric.MeterProvider
    tracer             trace.Tracer
    meter              metric.Meter
    resource            *resource.Resource
    logger             *zap.Logger

    llmTokensCounter    metric.Int64Counter
    llmLatencyHist      metric.Float64Histogram
}

```



```

    toolCallsCounter      metric.Int64Counter
    toolLatencyHist      metric.Float64Histogram
    agentIterCounter      metric.Int64Counter
    agentIterDurationHist metric.Float64Histogram

    stats TelemetryStats
    statsMu sync.RWMutex
    ringBuf *ringBuffer
}

func NewTelemetryProvider(ctx context.Context, cfg
    *TelemetryConfig, logger *zap.Logger)
    (*TelemetryProvider, error) {
    cfg.applyDefaults()
    if !cfg.Enabled {
        return newNoOpProvider(cfg, logger), nil
    }
    res, err := buildResource(ctx, cfg)
    if err != nil { return nil, err }

    var tracerExp sdktrace.SpanExporter
    var metricExp sdkmetric.Exporter
    switch cfg.Kind {
    case ExporterStdout:
        tracerExp, err =
            stdouttrace.New(stdouttrace.WithWriter(stdoutTraceWriter()))
        if err != nil { return nil, err }
        metricExp, err =
            stdoutmetric.New(stdoutmetric.WithWriter(stdoutMetricWriter()))
        if err != nil { return nil, err }
    case ExporterOTLPGRPC:
        tracerExp, err = otlptracegrpc.New(ctx,
            otlptracegrpc.WithEndpoint(cfg.Endpoint), ...)
        // ...
    case ExporterOTLPHTTP:
        // ...
    default:
        return nil, ErrUnknownExporterKind
    }

    tp := sdktrace.NewTracerProvider(
        sdktrace.WithBatcher(tracerExp),
        sdktrace.WithResource(res),
    )
    mp := sdkmetric.NewMeterProvider(
        sdkmetric.WithReader(sdkmetric.NewPeriodicReader(metricExp)),
        sdkmetric.WithResource(res),
    )

```

```

tracer := tp.Tracer("helixcode")
meter := mp.Meter("helixcode")

out := &TelemetryProvider{cfg: cfg, tracerProv: tp,
    meterProv: mp,
    tracer: tracer, meter: meter, resource: res, logger:
        logger}
if err := out.buildInstruments(); err != nil { return nil,
    err }
out.stats = TelemetryStats{Kind: cfg.Kind, Endpoint:
    cfg.Endpoint,
    ServiceName: cfg.ServiceName, StartTime: time.Now()}
if cfg.Kind == ExporterStdout {
    out.ringBuf = newRingBuffer(64)
}
return out, nil
}

func (t *TelemetryProvider) buildInstruments() error {
    var err error
    if t.llmTokensCounter, err =
        t.meter.Int64Counter("helixcode_llm_tokens_total"); err !=
        nil { return err }
    if t.llmLatencyHist, err =
        t.meter.Float64Histogram("helixcode_llm_latency_seconds");
        err != nil { return err }
    if t.toolCallsCounter, err =
        t.meter.Int64Counter("helixcode_tool_calls_total"); err !=
        nil { return err }
    if t.toolLatencyHist, err =
        t.meter.Float64Histogram("helixcode_tool_latency_seconds");
        err != nil { return err }
    if t.agentIterCounter, err =
        t.meter.Int64Counter("helixcode_agent_iterations_total");
        err != nil { return err }
    if t.agentIterDurationHist, err =
        t.meter.Float64Histogram("helixcode_agent_iteration_duration_seconds");
        err != nil { return err }
    return nil
}

func (t *TelemetryProvider) IsNoOp() bool { return t.cfg == nil
    || !t.cfg.Enabled }
func (t *TelemetryProvider) Tracer() trace.Tracer { return
    t.tracer }
func (t *TelemetryProvider) Meter() metric.Meter { return
    t.meter }

```

```

func (t *TelemetryProvider) Stats() TelemetryStats { /* RLock +
    return */ }
func (t *TelemetryProvider) RingBuffer() []ExportedRecord { /*
    nil-safe */ }
func (t *TelemetryProvider) ForceFlush(ctx context.Context)
    error { /* tp.ForceFlush + mp.ForceFlush */ }
func (t *TelemetryProvider) Shutdown(ctx context.Context) error
    { /* ForceFlush + Shutdown both */ }

```

Failing tests FIRST:

```

func TestNewTelemetryProvider_NoEnv_ReturnsNoOpProvider(t
    *testing.T) {
    cfg := &TelemetryConfig{Enabled: false}
    tp, err := NewTelemetryProvider(context.Background(), cfg,
        zap.NewNop())
    require.NoError(t, err)
    require.True(t, tp.IsNoOp())
    // Tracer is non-nil but emits nothing observable
    _, span := tp.Tracer().Start(context.Background(), "x")
    span.End()
}

func TestNewTelemetryProvider_StdoutKind_RealProvider(t
    *testing.T) {
    cfg := &TelemetryConfig{Enabled: true, Kind: ExporterStdout,
        ServiceName: "svc"}
    tp, err := NewTelemetryProvider(context.Background(), cfg,
        zap.NewNop())
    require.NoError(t, err)
    require.False(t, tp.IsNoOp())
    require.NotNil(t, tp.Tracer())
    require.NotNil(t, tp.Meter())
}

func TestProvider_ForceFlushDrainsBSP(t *testing.T) {
    // Use an in-process tracetest.SpanRecorder as the exporter:
    rec := tracetest.NewSpanRecorder()
    cfg := &TelemetryConfig{Enabled: true, Kind: ExporterStdout,
        ServiceName: "svc"}
    tp, _ :=
        newTelemetryProviderWithExporter(context.Background(),
            cfg, rec, zap.NewNop())
    _, span := tp.Tracer().Start(context.Background(), "test-
        span")
    span.End()
    require.NoError(t, tp.ForceFlush(context.Background()))
    spans := rec.Ended()
    require.Len(t, spans, 1)
}

```

```

    require.Equal(t, "test-span", spans[0].Name())
}

func TestProvider_MetricsForceFlushDrainsReader(t *testing.T) {
    // Manual reader pattern from go.opentelemetry.io/otel/sdk/
    // metric/metricdata
    reader := sdkmetric.NewManualReader()
    // ... construct provider with this reader, increment counter,
    // call collect, assert non-zero
}

func TestProvider_Shutdown_RespectsCtxDeadline(t *testing.T) {
    // Pass ctx with deadline already passed; ensure return <
    // 100ms.
}

```

Subject: feat(P1-F16-T05): TelemetryProvider with exporter selection + ForceFlush + Shutdown.

Task 6: llm_instrumentation.go (TDD)

Files: new helix_code/internal/telemetry/llm_instrumentation.go, new helix_code/internal/telemetry/llm_instrumentation_test.go.

TracedLLMProvider uses Go struct embedding so only Generate and GenerateStream are overridden; the other 9 methods of llm.Provider are promoted automatically.

```

type TracedLLMProvider struct {
    llm.Provider
    tp *TelemetryProvider
}

func NewTracedLLMProvider(inner llm.Provider, tp
    *TelemetryProvider) (*TracedLLMProvider, error) {
    if inner == nil { return nil, errors.New("telemetry: inner
        provider must not be nil") }
    return &TracedLLMProvider{Provider: inner, tp: tp}, nil
}

func (t *TracedLLMProvider) Generate(ctx context.Context, req
    *llm.LLMRequest) (*llm.LLMResponse, error) {
    if t.tp == nil || t.tp.IsNoOp() {
        return t.Provider.Generate(ctx, req)
    }
    start := time.Now()
    ctx, span := t.tp.Tracer().Start(ctx, "llm.generate",
        trace.WithAttributes(

```

```

        attribute.String("llm.model", req.Model),
        attribute.Int    ("llm.max_tokens", req.MaxTokens),
        attribute.Int    ("llm.message_count",
len(req.Messages)),
    ))
defer span.End()
resp, err := t.Provider.Generate(ctx, req)
dur := time.Since(start).Seconds()
if t.tp.llmLatencyHist != nil {
    t.tp.llmLatencyHist.Record(ctx, dur,
metric.WithAttributes(attribute.String("model",
req.Model)))
}
if resp != nil && t.tp.llmTokensCounter != nil {
    t.tp.llmTokensCounter.Add(ctx,
int64(resp.Usage.PromptTokens),
metric.WithAttributes(attribute.String("model",
req.Model), attribute.String("direction", "prompt")))
    t.tp.llmTokensCounter.Add(ctx,
int64(resp.Usage.CompletionTokens),
metric.WithAttributes(attribute.String("model",
req.Model), attribute.String("direction", "completion")))
    span.SetAttributes(
        attribute.Int("llm.usage.prompt_tokens",
resp.Usage.PromptTokens),
        attribute.Int("llm.usage.completion_tokens",
resp.Usage.CompletionTokens),
        attribute.Int("llm.usage.total_tokens",
resp.Usage.TotalTokens),
        attribute.String("llm.finish_reason",
resp.FinishReason),
    )
}
if err != nil {
    span.RecordError(err)
    span.SetStatus(codes.Error, err.Error())
}
return resp, err
}

func (t *TracedLLMProvider) GenerateStream(ctx context.Context,
req *llm.LLMRequest, ch chan<- llm.LLMResponse) error {
    if t.tp == nil || t.tp.IsNoOp() {
        return t.Provider.GenerateStream(ctx, req, ch)
    }
    // analogous to Generate; span ends after ch closes
}

```

Tests use a `tracetest.SpanRecorder`-backed `TelemetryProvider` and `subagent.FakeLLMProvider`:

```
func TestTracedLLMProvider_StartsSpan(t *testing.T) {
    rec := tracetest.NewSpanRecorder()
    tp := newTelemetryProviderForTest(t, rec)
    fake := subagent.NewFakeLLMProvider(map[string]string{"hi":
        "hello"})
    wrapped, err := NewTracedLLMProvider(fake, tp)
    require.NoError(t, err)
    _, err = wrapped.Generate(context.Background(),
        &llm.LLMRequest{Model: "fake-1", Messages:
            []llm.Message{{Role: "user", Content: "hi"}}})
    require.NoError(t, err)
    require.NoError(t, tp.ForceFlush(context.Background()))
    spans := rec.Ended()
    require.Len(t, spans, 1)
    require.Equal(t, "llm.generate", spans[0].Name())
}

func TestTracedLLMProvider_AddsTokensCounter(t *testing.T) {
    reader := sdkmetric.NewManualReader()
    tp := newTelemetryProviderWithMetricReader(t, reader)
    fake := subagent.NewFakeLLMProviderWithUsage("hi", "hello",
        llm.Usage{PromptTokens: 7, CompletionTokens: 3,
            TotalTokens: 10})
    wrapped, _ := NewTracedLLMProvider(fake, tp)
    _, _ = wrapped.Generate(context.Background(),
        &llm.LLMRequest{Model: "fake-1", Messages:
            []llm.Message{{Role: "user", Content: "hi"}}})
    var rm metricdata.ResourceMetrics
    require.NoError(t, reader.Collect(context.Background(), &rm))
    require.True(t, hasCounterValue(rm,
        "helixcode_llm_tokens_total",
        "model=fake-1,direction=prompt", 7))
    require.True(t, hasCounterValue(rm,
        "helixcode_llm_tokens_total",
        "model=fake-1,direction=completion", 3))
}

func TestTracedLLMProvider_DoesNotEmitPromptBodyAttribute(t
    *testing.T) {
    rec := tracetest.NewSpanRecorder()
    tp := newTelemetryProviderForTest(t, rec)
    wrapped, _ :=
        NewTracedLLMProvider(subagent.NewFakeLLMProvider(nil),
            tp)
```

```

_, _ = wrapped.Generate(context.Background(),
    &llm.LLMRequest{Model:"fake-1", Messages:
        []llm.Message{{Role:"user", Content:"SECRET-PROMPT-
        BODY"}}})
_ = tp.Flush(context.Background())
for _, span := range rec.Ended() {
    for _, kv := range span.Attributes() {
        k := strings.ToLower(string(kv.Key))
        require.NotContains(t, k, "prompt")
        require.NotEqual(t, "messages", k)
        require.NotContains(t, kv.Value.AsString(), "SECRET-
        PROMPT-BODY")
    }
}
}

func TestTracedLLMProvider_PassesThroughGetType(t *testing.T) {
    fake := subagent.NewFakeLLMProvider(nil)
    wrapped, _ := NewTracedLLMProvider(fake, nil)
    require.Equal(t, fake.GetType(), wrapped.GetType())
}

func TestTracedLLMProvider_NoOpFastPath_ZeroAllocs(t *testing.T)
{
    fake :=
        subagent.NewFakeLLMProvider(map[string]string{"hi":"ok"})
    cfg := &TelemetryConfig{Enabled: false}
    tp, _ := NewTelemetryProvider(context.Background(), cfg,
        zap.NewNop())
    wrapped, _ := NewTracedLLMProvider(fake, tp)
    avg := testing.AllocsPerRun(50, func() {
        _, _ = wrapped.Generate(context.Background(),
            &llm.LLMRequest{Model:"fake-1", Messages:
                []llm.Message{{Role:"user", Content:"hi"}}})
    })
    require.Less(t, avg, float64(2), "no-op path must not
        allocate per call")
}

func TestTracedLLMProvider_RecordsErrorOnFailure(t *testing.T) {
    rec := tracetest.NewSpanRecorder()
    tp := newTelemetryProviderForTest(t, rec)
    erroring := &erroringLLMProvider{err: errors.New("boom")}
    wrapped, _ := NewTracedLLMProvider(erroring, tp)
    _, err := wrapped.Generate(context.Background(),
        &llm.LLMRequest{Model:"x", Messages:
            []llm.Message{{Role:"user", Content:"x"}}})
    require.Error(t, err)
}

```

```

    require.NoError(t, tp.ForceFlush(context.Background()))
    spans := rec.Ended()
    require.Equal(t, codes.Error, spans[0].Status().Code)
}

```

Subject: feat(P1-F16-T06): TracedLLMProvider decorator + token counter + latency histogram + secret-attr safety.

Task 7: tool_instrumentation.go + registry wrap (TDD)

Files: new helix_code/internal/telemetry/tool_instrumentation.go, new helix_code/internal/telemetry/tool_instrumentation_test.go, modify helix_code/internal/tools/registry.go.

tool_instrumentation.go:

```

func InstrumentToolCall(ctx context.Context, tp
    *TelemetryProvider, toolName string) (context.Context,
    func(error)) {
    if tp == nil || tp.IsNoOp() {
        return ctx, func(error) {}
    }
    start := time.Now()
    ctx, span := tp.Tracer().Start(ctx, "tool.execute",
        trace.WithAttributes(attribute.String("tool.name",
            toolName)))
    return ctx, func(err error) {
        dur := time.Since(start).Seconds()
        status := "success"
        if err != nil {
            status = "error"
            span.RecordError(err)
            span.SetStatus(codes.Error, err.Error())
        }
        if tp.toolCallsCounter != nil {
            tp.toolCallsCounter.Add(ctx, 1,
                metric.WithAttributes(attribute.String("tool",
                    toolName), attribute.String("status", status)))
        }
        if tp.toolLatencyHist != nil {
            tp.toolLatencyHist.Record(ctx, dur,
                metric.WithAttributes(attribute.String("tool",
                    toolName)))
        }
        span.End()
    }
}

```


registry.go modifications:

```
import "dev.helix.code/internal/telemetry"

// add field
telemetryProvider *telemetry.TelemetryProvider

// new setter
func (r *ToolRegistry) SetTelemetryProvider(tp
    *telemetry.TelemetryProvider) {
    r.mu.Lock(); defer r.mu.Unlock()
    r.telemetryProvider = tp
}

// inside Execute, after tool/Validate checks, before fireBefore:
ctx, end := telemetry.InstrumentToolCall(ctx,
    r.telemetryProvider, name)
var execErr error
defer func() { end(execErr) }()
// existing fireBefore + tool.Execute (assigns to result,
    execErr) + fireAfter chain unchanged
```

Tests:

```
func TestInstrumentToolCall_NilProvider_NoOp(t *testing.T) {
    ctx, end := InstrumentToolCall(context.Background(), nil,
        "bash")
    require.NotNil(t, ctx)
    require.NotPanics(t, func() { end(nil) })
}

func TestInstrumentToolCall_SuccessPath_RecordsSuccessLabel(t
    *testing.T) {
    rec := tracetest.NewSpanRecorder()
    reader := sdkmetric.NewManualReader()
    tp := newTelemetryProviderForTest(t, rec, reader)
    _, end := InstrumentToolCall(context.Background(), tp,
        "bash")
    end(nil)
    require.NoError(t, tp.ForceFlush(context.Background()))
    spans := rec.Ended()
    require.Len(t, spans, 1)
    require.Equal(t, "tool.execute", spans[0].Name())
    require.Equal(t, codes.Unset, spans[0].Status().Code)
    var rm metricdata.ResourceMetrics
    _ = reader.Collect(context.Background(), &rm)
    require.True(t, hasCounterValue(rm,
        "helixcode_tool_calls_total",
        "tool=bash,status=success", 1))
}
```

```

}

func TestInstrumentToolCall_ErrorPath_RecordsErrorAndStatus(t
    *testing.T) { /* end(boom); status code Error */ }

func TestRegistry_Execute_EmitsToolSpan(t *testing.T) {
    // Real registry + a no-op echo tool + tracetest.SpanRecorder
    TelemetryProvider
    // After Execute, span with name=tool.execute and
    tool.name=echo exists
}

```

Subject: feat(P1-F16-T07): InstrumentToolCall helper +
ToolRegistry.Execute span/counter/histogram in-place.

Task 8: agent_instrumentation.go + base_agent wrap (TDD)

Files: new helix_code/internal/telemetry/agent_instrumentation.go, new helix_code/internal/telemetry/agent_instrumentation_test.go, modify helix_code/internal/agent/base_agent.go.

agent_instrumentation.go:

```

func InstrumentAgentIteration(ctx context.Context, tp
    *TelemetryProvider) (context.Context, func(error)) {
    if tp == nil || tp.IsNoOp() {
        return ctx, func(error) {}
    }
    start := time.Now()
    ctx, span := tp.Tracer().Start(ctx, "agent.iteration")
    return ctx, func(err error) {
        dur := time.Since(start).Seconds()
        if err != nil {
            span.RecordError(err)
            span.SetStatus(codes.Error, err.Error())
        }
        if tp.agentIterCounter != nil {
            tp.agentIterCounter.Add(ctx, 1)
        }
        if tp.agentIterDurationHist != nil {
            tp.agentIterDurationHist.Record(ctx, dur)
        }
        span.End()
    }
}

```

base_agent.go modifications:

```

// new field
telemetryProvider *telemetry.TelemetryProvider

// new setter
func (a *BaseAgent) SetTelemetryProvider(tp
    *telemetry.TelemetryProvider) {
    a.telemetryProvider = tp
}

// inside executeTaskWithLLM, immediately before the existing
`response, err := a.llmProvider.Generate(...)`
ctx, endIter := telemetry.InstrumentAgentIteration(ctx,
    a.telemetryProvider)
var iterErr error
defer func() { endIter(iterErr) }()
// existing Generate call assigns to response, err
// after processLLMResponse:
if err != nil { iterErr = err }

```

Tests:

```

func TestInstrumentAgentIteration_NilProvider_NoOp(t *testing.T)
    { /* ... */ }

func TestInstrumentAgentIteration_RecordsDuration(t *testing.T) {
    rec := tracetest.NewSpanRecorder()
    reader := sdkmetric.NewManualReader()
    tp := newTelemetryProviderForTest(t, rec, reader)
    _, end := InstrumentAgentIteration(context.Background(), tp)
    time.Sleep(2*time.Millisecond)
    end(nil)
    require.NoError(t, tp.ForceFlush(context.Background()))
    spans := rec.Ended()
    require.Equal(t, "agent.iteration", spans[0].Name())
    var rm metricdata.ResourceMetrics
    _ = reader.Collect(context.Background(), &rm)
    require.True(t, hasCounterValueAtLeast(rm,
        "helixcode_agent_iterations_total", "", 1))
}

func TestBaseAgent_executeTaskWithLLM_EmitsAgentIterationSpan(t
    *testing.T) {
    // Construct BaseAgent with FakeLLMProvider + tracetest
    TelemetryProvider
    // Run executeTaskWithLLM; assert span named agent.iteration
    exists
}

```

Subject: feat(P1-F16-T08): InstrumentAgentIteration helper + BaseAgent.executeTaskWithLLM span/counter/duration in-place.

Task 9: /telemetry slash command (TDD)

Files: new helix_code/internal/commands/telemetry_command.go, new helix_code/internal/commands/telemetry_command_test.go.

Mirrors F14 /sandbox and F15 /subagents shape: defines TelemetryProviderInspector interface in the commands package so the slash is testable with a fake.

```
type TelemetryProviderInspector interface {
    Stats() telemetry.TelemetryStats
    RingBuffer() []telemetry.ExportedRecord
    ForceFlush(ctx context.Context) error
    IsNoOp() bool
}

type TelemetryCommand struct { tp TelemetryProviderInspector }

func NewTelemetryCommand(tp TelemetryProviderInspector)
    *TelemetryCommand
func (c *TelemetryCommand) Name() string { return "telemetry" }
func (c *TelemetryCommand) Execute(ctx context.Context, cc
    *CommandContext) (*CommandResult, error) {
    if c.tp == nil { return &CommandResult{Success: false,
        Output: "telemetry unavailable"}, nil }
    sub := "status"
    if len(cc.Args) > 0 { sub = cc.Args[0] }
    switch sub {
    case "status": return c.handleStatus(), nil
    case "show":   return c.handleShow(), nil
    case "flush":  return c.handleFlush(ctx), nil
    default:       return &CommandResult{Success: false, Output:
        c.Usage()}, nil
    }
}
```

Tests:

```
func TestTelemetryCommand_Status_RendersTable(t *testing.T) { /*
    fake with Kind=stdout; output contains KIND, ENDPOINT,
    SERVICE, SPANS, METRICS */ }
func TestTelemetryCommand_Status_NoOp_ReportsDisabled(t
    *testing.T) { /* IsNoOp=true; output contains "telemetry
    disabled" + the env-var hint */ }
```

```

func TestTelemetryCommand_Show_StdoutOnly(t *testing.T) { /*
    RingBuffer returns 2 records; output contains both names
    */ }
func TestTelemetryCommand_Show_NonStdout_ReportsRingBufferEmpty(t
    *testing.T) { /* fake.IsNoOp=false but RingBuffer is empty
    (OTLP) */ }
func TestTelemetryCommand_Flush_DelegatesToProviderForceFlush(t
    *testing.T) { /* fake.flushCalled=true */ }
func TestTelemetryCommand_Flush_DeadlineErrorReportedToUser(t
    *testing.T) { /* fake returns context.DeadlineExceeded →
    output contains "flush timed out" */ }
func TestTelemetryCommand_NilProvider_ReportsUnavailable(t
    *testing.T) { /* nil tp → output "telemetry unavailable"
    */ }

```

Subject: feat(P1-F16-T09): /telemetry slash command (status/show/flush) + nil-provider unavailable.

Task 10: main.go wiring + integration tests

Files: modify `helix_code/cmd/cli/main.go`, new `helix_code/tests/integration/telemetry_test.go` (//go:build integration).

`main.go` additions (per spec §4.1) — three blocks alongside the existing logger construction:

```

// 1. Load + construct
telCfg, err := telemetry.LoadConfigFromEnv()
if err != nil {
    logger.Warn("telemetry config invalid; continuing without
        telemetry", zap.Error(err))
    telCfg = &telemetry.TelemetryConfig{Enabled: false, Kind:
        telemetry.ExporterNone}
}
tp, err := telemetry.NewTelemetryProvider(ctx, telCfg, logger)
if err != nil {
    logger.Warn("telemetry init failed; continuing without
        telemetry", zap.Error(err))
    tp = nil
}
defer func() {
    if tp == nil { return }
    shCtx, sc := context.WithTimeout(context.Background(),
        5*time.Second)
    defer sc()
    if err := tp.Shutdown(shCtx); err != nil {
        logger.Warn("telemetry shutdown failed", zap.Error(err))
    }
}()

```

```
// 2. Decorate the LLM provider (after factory builds it)
if tp != nil && !tp.IsNoOp() {
    if wrapped, werr := telemetry.NewTracedLLMProvider(provider,
        tp); werr == nil {
        provider = wrapped
    } else {
        logger.Warn("telemetry: failed to wrap LLM provider",
            zap.Error(werr))
    }
}
}
```

```
// 3. Setters on registry + agent + slash registration
if tp != nil {
    toolReg.SetTelemetryProvider(tp)
    baseAgent.SetTelemetryProvider(tp) // wherever the
        BaseAgent is constructed
}
slashRegistry.Register(commands.NewTelemetryCommand(tp))
```

Integration tests (gated):

```
//go:build integration
// +build integration
```

```
func TestTelemetry_StdoutExporter_RealLLMCall_FakeProvider(t
    *testing.T) {
    t.Setenv("OTEL_EXPORTER_OTLP_PROTOCOL", "stdout")
    var buf bytes.Buffer
    cfg, _ := telemetry.LoadConfigFromEnv()
    tp, err :=
        telemetry.NewTelemetryProviderWithStdoutWriters(context.Background(),
            cfg, zap.NewNop(), &buf, &buf)
    require.NoError(t, err)
    fake :=
        subagent.NewFakeLLMProvider(map[string]string{"hi": "ok"})
    wrapped, _ := telemetry.NewTracedLLMProvider(fake, tp)
    _, err = wrapped.Generate(context.Background(),
        &llm.LLMRequest{Model: "fake-1", Messages:
            []llm.Message{{Role: "user", Content: "hi"}}})
    require.NoError(t, err)
    require.NoError(t, tp.ForceFlush(context.Background()))
    out := buf.String()
    require.Contains(t, out, `Name:"llm.generate"`)
    require.Contains(t, out, "helixcode_llm_tokens_total")
}
```

```
func TestTelemetry_OTLPGRPC_RealCollector(t *testing.T) {
    if os.Getenv("OTEL_EXPORTER_OTLP_ENDPOINT") == "" {
```

```

        t.Skip("SKIP-OK: P1-F16 OTEL_EXPORTER_OTLP_ENDPOINT unset
        (export OTEL_EXPORTER_OTLP_ENDPOINT=localhost:4317
        OTEL_EXPORTER_OTLP_PROTOCOL=grpc)")
    }
    if os.Getenv("OTEL_EXPORTER_OTLP_PROTOCOL") != "grpc" {
        t.Skip("SKIP-OK: P1-F16 OTEL_EXPORTER_OTLP_PROTOCOL not
        'grpc'")
    }
    cfg, err := telemetry.LoadConfigFromEnv()
    require.NoError(t, err)
    tp, err :=
        telemetry.NewTelemetryProvider(context.Background(),
            cfg, zap.NewNop())
    require.NoError(t, err)
    _, span := tp.Tracer().Start(context.Background(), "test-
    export-grpc"); span.End()
    require.NoError(t, tp.ForceFlush(context.Background()))
    require.NoError(t, tp.Shutdown(context.Background()))
}

func TestTelemetry_OTLPHTTP_RealCollector(t *testing.T) {
    if os.Getenv("OTEL_EXPORTER_OTLP_ENDPOINT") == "" {
        t.Skip("SKIP-OK: P1-F16 OTEL_EXPORTER_OTLP_ENDPOINT unset
        (export OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4318
        OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf)")
    }
    if os.Getenv("OTEL_EXPORTER_OTLP_PROTOCOL") != "http/
    protobuf" {
        t.Skip("SKIP-OK: P1-F16 OTEL_EXPORTER_OTLP_PROTOCOL not
        'http/protobuf'")
    }
    // analogous to gRPC test
}

func TestTelemetry_ToolRegistry_InstrumentedExecute(t
    *testing.T)
    { /* stdout + real tool + assert span/counter */ }
func TestTelemetry_AgentIteration_RealLLMCall_StdoutExport(t
    *testing.T) { /* stdout + BaseAgent + FakeLLMProvider
    */ }
func TestTelemetry_NoOp_FastPath_RealRegistry(t *testing.T) { /*
    1000 calls; latency within 5% baseline */ }

```

Subject: feat(P1-F16-T10): wire TelemetryProvider into main.go + /
telemetry + gated integration tests.

Task 11: Challenge with in-tree fake OTLP/HTTP receiver

Files: new helix_code/tests/integration/cmd/p1f16_challenge/main.go, new challenges/p1-f16-opentelemetry-integration/CHALLENGE.md, new challenges/p1-f16-opentelemetry-integration/run.sh.

Harness phases (per spec §6.3): 1. **STDOUT (always runs)** — captures the stdout exporter's writer, runs LLM + tool + agent through the full instrumentation path with FakeLLMProvider, asserts "Name": "llm.generate", "Name": "tool.execute", "Name": "agent.iteration" and helixcode_llm_tokens_total in the captured bytes; asserts ForceFlush returns within 5 s. 2. **FAKE OTLP/HTTP RECEIVER (always runs)** — starts an in-tree net/http.Server on 127.0.0.1:0 registering POST /v1/traces and POST /v1/metrics. Each handler decodes the OTLP protobuf body using go.opentelemetry.io/proto/otlp/collector/trace/v1+.../collector/metrics/v1, increments a per-endpoint counter, returns 200 OK. Harness sets OTEL_EXPORTER_OTLP_ENDPOINT=http://127.0.0.1:<port> + OTEL_EXPORTER_OTLP_PROTOCOL=http/protobuf, runs the same workflow, force-flushes. Asserts: receiver got ≥ 1 trace POST AND ≥ 1 metric POST; the decoded trace body contains a span named llm.generate. **Anti-bluff: real export, real HTTP, real protobuf.** 3. **SECRET-ATTRIBUTE FILTER (always runs)** — deliberately injects a span attribute api_key=sk-abc-123 via the FilterAttributes seam; deliberately injects prompt=<long body>; asserts the captured stdout export does NOT contain those keys/values. Asserts safe attributes (model, tool.name) survive. 4. **NO-OP FAST PATH (always runs)** — disables telemetry; runs 1000 tool calls; asserts wall-clock latency within 5% of the un-instrumented baseline (re-runs without the wrap to derive the baseline). 5. **REAL OTLP COLLECTOR (gated)** — OTEL_EXPORTER_OTLP_ENDPOINT + OTEL_EXPORTER_OTLP_PROTOCOL=grpc set $\rightarrow$ connects to a real collector, exports a span, asserts Shutdown returns no error. Otherwise: [skipped: OTEL_EXPORTER_OTLP_ENDPOINT unset (export OTEL_EXPORTER_OTLP_ENDPOINT=...)].

Output skeleton (verbatim per spec §6.3) ends with:

SUMMARY: STDOUT=6/6 PASS; OTLP-HTTP=5/5 PASS; FILTER=3/3 PASS; NOOP=2/2 PA

The Challenge MUST exit non-zero on any assertion failure within phases that did run. Span-count-zero is treated as a hard failure. Anti-bluff smoke clean check appended to output. Verbatim output captured into 06_phase_1_evidence.md. Dual commit (Challenges submodule + meta-repo bump).

Subject: feat(P1-F16-T11): challenge with runtime evidence (stdout + fake OTLP/HTTP receiver always; real-collector gated).

Task 12: Close-out + push

Tick all 12 items in PROGRESS, advance PROGRESS focus to F17 candidate, run final verification:

```
cd HelixCode && make verify-compile
grep -rn "simulated\|for now\|TODO implement\|placeholder" \
    internal/telemetry internal/commands/telemetry_command.go &&
    echo BLUFF || echo clean
```



```
go test -count=1 ./internal/telemetry/... ./internal/commands/...
go test -count=1 -tags=integration ./tests/integration/...
```

Commit chore(P1-F16-T12): close out feature 16 – opentelemetry integration. Push 4 remotes non-force (origin, helixdev, vasic-digital, gitlab per programme conventions). Request explicit user authorization at this step (CONST-043).

Self-review notes

1. **Spec coverage:** every spec section maps to a task — T02 deps (§3.5), T03 types + DefaultBlockedAttributeKeys (§3.3, §5.2), T04 config + filter (§3.3, §3.4, §5.2 CONST-042), T05 provider (§3.3, §4.5), T06 TracedLLMProvider (§4.2, §5.2 criteria 1+3+5), T07 tool instrumentation (§4.3, §5.2 criteria 1+3), T08 agent instrumentation (§4.4), T09 slash (§3.4), T10 main.go wiring + integration (§4.1, §6.2), T11 Challenge five phases (§5.2, §6.3), T12 close-out (§9).
2. **TDD:** every code task starts with a failing test that exercises real OTEL SDK paths — the deps-link smoke (T02), the env-var cases (T04), `tracetest.SpanRecorder` + manual metric reader (T05/T06), the in-place wrap assertions (T07/T08), the slash with fake inspector (T09).
3. **Type consistency:** `TelemetryConfig`, `ExporterKind`, `TelemetryProvider`, `TracedLLMProvider`, `InstrumentToolCall`, `InstrumentAgentIteration`, `TelemetryProviderInspector`, `DefaultBlockedAttributeKeys`, env-var names — all match across spec §3.3 and plan T03–T09.
4. **New external deps:** all OTEL modules pinned to v1.30.0 (synchronised release train); no CGO; transitive footprint estimated at ~25 modules in `go.sum`. T02 runs `go mod tidy` and reviews the diff for spurious major-version drift before commit.
5. **Anti-bluff (§5.2):** Challenge has FIVE phases; `STDOUT`, `FAKE-OTLP-HTTP`, `FILTER`, and `NOOP` always run; `REAL-COLLECTOR` is gated on `OTEL_EXPORTER_OTLP_ENDPOINT`. The five real-execution criteria (spans created, spans exported, metrics recorded, metrics exported, no secret-shaped attrs) each have a dedicated PASS line. Span-count-zero is a hard Challenge failure — absence-of-error is NEVER acceptable.
6. **CONST-042:** `DefaultBlockedAttributeKeys` covers credential keys + prompt-body keys. `FilterAttributes` is case-insensitive. Prompt body is NEVER added as a span attribute by `TracedLLMProvider`. `TestTracedLLMProvider_DoesNotEmitPromptBodyAttribute` enforces this. The Challenge's `FILTER` phase deliberately injects forbidden attributes and asserts they are dropped.
7. **CONST-043:** stays on `main`, non-force to all four remotes; explicit user authorization is requested at T12 before pushing.
8. **Decorator vs in-place — non-obvious call** (recorded in spec §11): `TracedLLMProvider` uses Go struct embedding so adding a method to `llm.Provider` does not require updating the wrapper. The tool registry and agent loop use in-place instrumentation (5 added lines each) because they are already chokepoints with multiple existing pre/post hooks; a parallel decorator hierarchy would force every test that exercises the registry to thread the decorator through.
9. **Sampling default — non-obvious call** (recorded in spec §11): left at OTEL SDK default parent-based(`traces-and-error-codes`). Users who want head sampling set `OTEL_TRACES_SAMPLER` per the OTEL spec; the SDK reads it automatically. F16 does not pin a sampler.

10. **Subagent helpers not instrumented in v1** (deferred to F16.5): the subagent subprocess helper-mode constructs a minimal runtime that doesn't share parent state. Threading the parent's TracerProvider through the env-var payload is a non-trivial protocol change; documented loudly in spec §8 and §5.2.
11. **No-op fast path is a real test target** (not just a comment):
TestTracedLLMProvider_NoOpFastPath_ZeroAllocs uses `testing.AllocsPerRun` to assert <2 allocations per call when telemetry is disabled. The Challenge's NOOP phase asserts 1000 tool calls stay within 5% of the un-instrumented baseline.
12. **Span-attribute naming** — OTEL semconv keys where they exist (`llm.model`, `tool.name`); `helixcode_*` snake_case prefix for metric names so the OTEL-to-Prometheus translator preserves a HelixCode namespace.
13. **Why no logs-bridge in v1** — OTEL Go logs API entered Beta in 2024; we wait for stable. zap stays the logging surface. Deferred to F16.5.
14. **Why no Prometheus exporter** — OTLP + the OpenTelemetry Collector covers the Prometheus consumer end-to-end without HelixCode owning a second exporter chain. Explicitly rejected.